VisionPlay Project Report

Webcam based rock paper scissors using computer vision and machine learning

Prepared for a computer vision project review

Team member 1 Ashish Ranjan 235805340
Team member 2 Vishnu Nair 235805442
Class or course Computer Vision

Abstract

VisionPlay is an educational desktop application that turns hand gestures from a webcam into moves in a rock paper scissors game. It combines OpenCV for image capture and display, a pretrained MediaPipe Gesture Recognizer for hand analysis, and an optional support vector machine trained from the user's own labeled hand landmarks. Both recognition paths depend on the same pretrained hand detection and landmark extraction; the project does not train a neural network from scratch.

The custom classifier uses 21 three dimensional landmarks, converts them into a wrist relative and scale normalized vector of 63 features, and applies a StandardScaler and radial basis function SVM. A guided workflow collects separate capture sessions. The newest complete session is held out for evaluation, reducing leakage from neighboring video frames. The game commits the computer's random move before the countdown and accepts the player's move only after a temporal stability check.

The implementation supports local operation, explicit training controls, an explanatory teacher view, and cancellation without scoring when a valid stable move is unavailable. The saved local dataset has 720 landmark samples in six complete sessions. Its evaluation record reports 120 correct classifications among 120 examples from the newest held-out session. The CSV counts and recorded confusion matrix are internally consistent, but the result is limited to this local session split; no independent participant study, pretrained-path accuracy test, or controlled speed benchmark is documented. [3,6,14]

Keywords Computer vision, gesture recognition, MediaPipe, hand landmarks, support vector machine, session based evaluation.

Source basis Repository revision 415ef64, reviewed 27 October 2026. This report describes that revision; later changes may differ. [1–6]

1 Introduction and objectives

Problem statement

A keyboard or mouse supplies a discrete game choice directly. A webcam supplies images, so the application must infer an intended move from a continuous and sometimes ambiguous signal. Hands move, lighting changes, fingers overlap, and a detected pose can fluctuate between frames. A classroom project must make that recognition pipeline understandable while preventing uncertain observations from producing arbitrary game results.

VisionPlay addresses this problem through a small, visible task: the player shows a fist for rock, an open palm for paper, or a V sign for scissors. The game is simple enough that the teacher can concentrate on the computer vision and learning stages rather than complex game rules. The project also separates gesture classification from the computer opponent, which chooses randomly and does not learn a playing strategy. [1,2,5]

Project objectives

- Convert webcam frames into a live hand representation and show the detected landmark structure to the user.
- Provide a usable pretrained recognition mode before collecting any personal training examples.
- Train a classical three class classifier from correctly labeled landmark samples and evaluate it using a separate capture session.
- Make rounds predictable and fair through early computer move commitment, temporal validation, and cancellation when evidence is insufficient.
- Explain the roles of computer vision, pretrained deep learning, supervised learning, and ordinary game logic in one reproducible application.

Scope of the implementation

The current scope is one detected hand and three static gestures on a local desktop. The optional SVM recognizes hand shape; it does not infer intention beyond those classes. The system does not implement sign language translation, body tracking, speech interaction, player identification, online multiplayer, or an adaptive opponent. These exclusions keep the demonstration focused and limit the claims that can reasonably be made from it.

Report organization

Sections 2 to 5 explain the technical basis, architecture, feature representation, and classifiers. Sections 6 and 7 describe data collection and evaluation. Sections 8 to 10 cover round behavior, the interface, setup, and privacy. Sections 11 and 12 distinguish software verification, the saved local evaluation, and measurements still needed. Section 13 presents limitations, future work, and the conclusion.

Success is defined here as a transparent end to end demonstration with defensible data handling and game behavior. The recorded session-holdout result is reported with its sample count and scope; no general accuracy target or cross-user performance claim is inferred from it.

2 Technical background and design choices

Computer vision and pretrained deep learning

Computer vision is the process of extracting useful information from images. In VisionPlay, OpenCV opens the camera, prepares image frames, draws the hand skeleton, and renders the interactive window. The pretrained MediaPipe task performs the learned hand analysis. Its outputs include landmarks and gesture categories; the application uses the image coordinate landmarks rather than the world coordinate output. [2,7,8,12]

The distinction matters because the project combines software layers with different responsibilities. Mirroring an image and drawing a line are image processing operations. Finding the hand and estimating its landmarks use pretrained neural models. A lookup that maps Closed_Fist to rock is ordinary application logic. None of those mappings represents training a new neural network.

Why landmarks are used as features

A raw camera image contains background, clothing, illumination, and many pixels that are unrelated to finger shape. A landmark vector is a compact representation of the hand geometry. The project uses that representation to make a small personal supervised learning experiment practical. The approach also creates an understandable visual connection between the skeleton shown on screen and the numbers passed to the classifier.

This choice does not remove dependence on the pretrained model. If the landmark detector misses a finger, loses the hand, or estimates a misleading pose, the SVM receives imperfect inputs. The two recognition paths therefore share a source of error. They must not be presented as independent raw image models or as a controlled deep learning versus classical learning benchmark. [2,3]

Design alternatives and selected approach

Approach	Educational benefit	Main tradeoff
Direct gesture labels	Usable immediately after setup	Limited to supported labels and model behavior
Landmark based SVM	Shows labeling, fitting and evaluation	Needs personal samples; depends on landmark quality
New raw image neural model	Could study image model training	Outside scope; would require a separate dataset and training study

The implemented design includes the first two approaches only. It uses the pretrained path as the initial interaction mode and the SVM path to demonstrate supervised learning. The application is not a hand engineered finger counting recognizer, and it has no rule based recognition fallback. Game rules and stability rules should not be confused with an alternative gesture model. [1–5]

3 System architecture

The application processes each camera frame locally and produces a candidate gesture, then separates recognition from round resolution. The active mode determines which classifier supplies the candidate. A shared stability gate decides whether recent candidates are consistent enough to use. The game state compares the accepted move with the computer move that was already committed. [2,5]

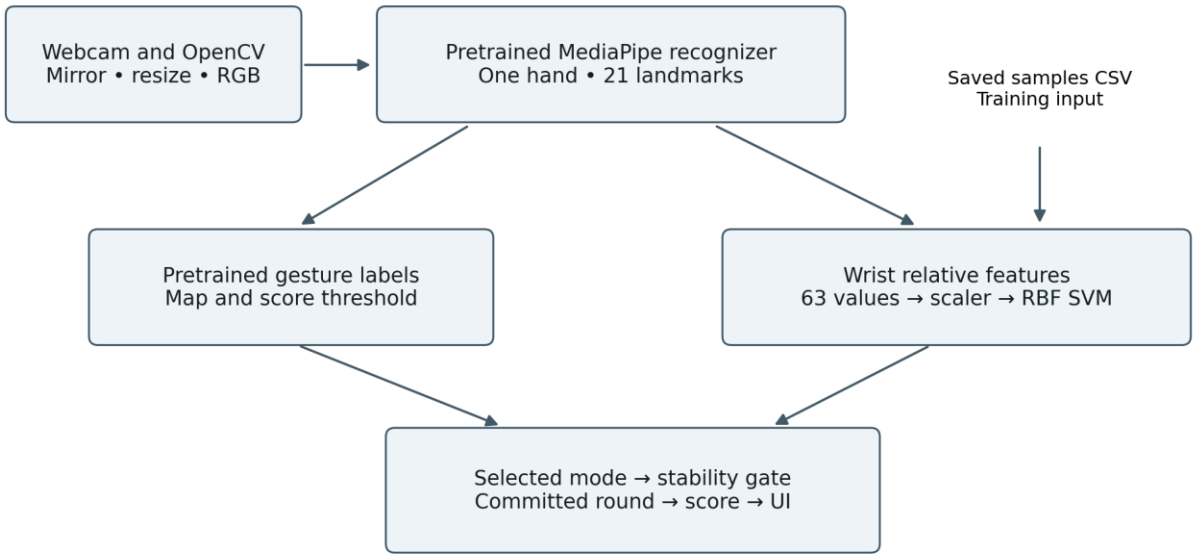


Figure 1 Processing architecture derived from the source code. This is an original explanatory diagram, not a runtime screenshot.

Main implementation modules

Module	Responsibility
app.py	Camera lifecycle, recognizer, per frame processing and action dispatch
core.py	Feature normalization, class labels and rock paper scissors outcome
learning.py	CSV validation, fitting, session holdout and evaluation report
training_state.py	Guided capture stages, timing, progress and pause behavior
controller.py and game_state.py	Screen actions, stability, round phases and match scoring
ui.py	Game, training and teacher screen rendering and button regions

The separation makes game and training transitions testable without a real camera. Such tests verify software logic, not gesture recognition accuracy. A model output becomes a game decision only through the state and timing rules; the interface is not allowed to choose a new computer response after seeing the accepted player move. [2–6]

4 Image processing and feature extraction

Frame preparation

OpenCV captures a frame, mirrors it horizontally, and resizes it to a width of 640 pixels while preserving its aspect ratio. The frame is converted from OpenCV's BGR channel order to RGB for MediaPipe. The recognizer runs in VIDEO mode with strictly increasing millisecond timestamps. Although the input comes from a webcam, the source uses the video call rather than the asynchronous LIVE_STREAM interface. [2,7]

The recognizer is configured for one hand. Detection, hand presence, and tracking confidence thresholds are each set to 0.60. These are acceptance settings at different stages, not measured success rates. A missing hand produces no candidate move and no training sample.

Landmark representation

Each detected hand provides 21 landmarks with x, y, and z coordinates. The feature vector therefore has 21 × 3 = 63 values. Coordinates come from normalized image landmarks. The depth component is an estimated relative value, not a physical distance measured by a depth camera. VisionPlay does not use world landmarks to claim distances in centimeters. [2,8]

The feature function first subtracts the wrist, landmark 0, from every landmark. It then divides all coordinates by the largest Euclidean distance from that wrist origin and flattens the ordered coordinates. For landmark i, the transformation is: [2]

$$q_i = p_i - p_0$$

$$s = max(\|q_0\|, \dots, \|q_{20}\|)$$

$$f = (\frac{q_{0x}}{s}, \frac{q_{0y}}{s}, \frac{q_{0z}}{s}, \dots, \frac{q_{20z}}{s})$$

Here p is an input landmark, q is its wrist relative coordinate, s is the maximum wrist distance, and f concatenates the normalized x, y, and z components in landmark order. Invalid coordinates or a degenerate scale are rejected rather than used in a division by zero.

Effect and remaining limitations

Wrist subtraction reduces sensitivity to the hand's image position. Scaling reduces sensitivity to apparent hand size in this landmark representation. The same landmark ordering must be used during capture, fitting, and prediction. The wrist entries become zero, but remain part of the fixed 63 value schema.

This transformation does not align the hand to a canonical rotation or standardize left and right hands. Viewpoint, finger occlusion, perspective, and landmark errors can still change the representation. Initially using the same hand with its palm facing the camera is therefore a reasonable demonstration constraint; broader robustness requires separate testing rather than an invariance claim.

5 Classification methods

Pretrained recognition path

The default path reads the top MediaPipe gesture label and score. Closed_Fist maps to rock, Open_Palm maps to paper, and Victory maps to scissors. The top label must be supported by this mapping and have a score of at least 0.60. Otherwise the frame supplies no valid game candidate. This path works without a personal training dataset after the official model has been downloaded. [1,2,7,8]

A live score describes one prediction from the pretrained model. It is not the percentage of correctly classified gestures in a project test. The application does not independently measure pretrained recognition accuracy on a labeled holdout set. The temporal stability bar is another distinct quantity and should not be described as a confidence probability.

Classical supervised learning path

The optional path predicts from the normalized 63 feature vector. A scikit-learn pipeline applies StandardScaler, then SVC with an RBF kernel. StandardScaler learns a mean and scale for each feature from the fitting rows, then applies the same transformation to new inputs. This is separate from the per hand geometric normalization described in Section 4. [3,9,10]

The RBF kernel allows a nonlinear decision boundary in the feature space. C controls the regularization tradeoff and gamma controls the reach of individual examples in the kernel. The chosen values are fixed project settings, not values demonstrated to be optimal through a hyperparameter search. Multiclass prediction is handled by the SVC implementation. [3,9]

Setting	Implemented value
Input representation	63 normalized landmark coordinates
Preprocessing	StandardScaler fitted within the pipeline
Estimator	SVC with RBF kernel
C and gamma	3.0 and scale
Output classes	rock, paper, scissors
Unknown rejection and probability	No unknown class and no displayed SVM probability

When a hand is detected, the trained SVM chooses one of its three classes even if the pose is unfamiliar. A stable but incorrect sequence can still pass the temporal check. The model does not choose the computer move, learn game strategy, or adapt automatically after a win or loss. Personal samples and labels are the only basis of this supervised gesture fitting step. [2,3]

6 Dataset collection methodology

Labels and capture sessions

Guided training asks for rock, paper, and scissors in that order. The label is supplied by the requested pose, not copied from a prediction. Showing paper while the wizard asks for rock creates a mislabeled example even if the camera tracks the hand correctly. The collector stores a session identifier, a label, and 63 feature columns in data/samples.csv. [3,4]

Each new batch targets 40 detected hand samples. Capture begins after a two second preparation period and records at most one observed sample every 0.20 seconds. It does not generate synthetic catch up frames if processing is slow. Forty samples take roughly eight seconds of uninterrupted visible hand capture, plus preparation and interruptions; this is a timing estimate, not a measured throughput result.

Reproducible collection procedure

1. Open Guided training with G. Use one hand fully in view and follow the requested pose.
2. Press Capture or Space or C. Keep the requested class fixed during the batch, with small natural variation in position.
3. Explicitly start the next pose when ready. The wizard does not automatically capture a different label while the user is changing hand shape.
4. After all three poses, reposition the hand and vary distance or lighting moderately before explicitly starting a new session.
5. Complete at least three sessions, stop active capture, and use Train or T to fit and evaluate the classifier.

Planned capture counts and actual saved data

Collection plan	Conditional example
Samples per pose per new session	40
Poses and new sessions	3 poses across 3 sessions
Total if all nine batches finish	360 samples
Evaluation split in that exact case	240 fit samples and 120 held out samples

These counts describe a three-session example, not the current saved dataset. The local CSV contains six complete sessions, each with 40 rows per class, for 720 rows total. Existing or interrupted data can produce different totals. For compatibility, the evaluator accepts a complete session when every class has at least 20 samples, while new capture aims for 40. Incomplete sessions are excluded from fitting and evaluation; none are present in this saved dataset. [3,4,14]

Cancel or Back pauses capture without deleting saved rows. The running application can resume its current batch. Restarting preserves the CSV but creates a fresh session identifier. A batch pauses after 30 seconds beyond preparation if it has not completed, rather than inventing missing observations.

7 Evaluation design and metrics

Why sessions are held out

Adjacent frames in one recording can be almost identical. Randomly placing neighboring frames in both fitting and test sets can make the result look stronger than performance on a fresh recording. VisionPlay groups examples by capture session. It requires at least three complete sessions and holds out the newest complete session according to its first appearance in CSV order. Earlier complete sessions fit the evaluation pipeline. [3]

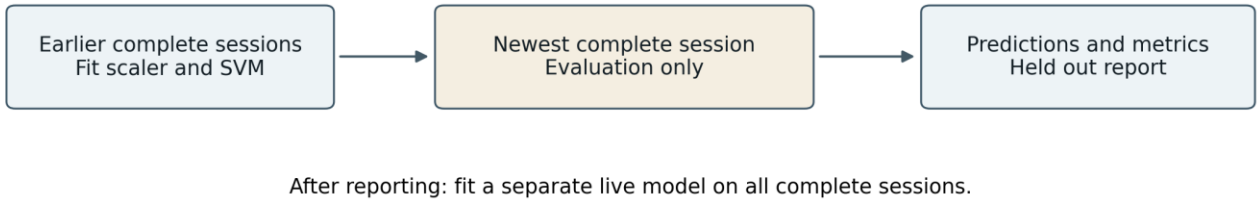


Figure 2 Session based evaluation and later refitting. Arrows show the procedure, not measured sample counts.

The scaler is fitted inside the evaluation pipeline using only the fitting subset. Test rows do not set its scaling statistics. The code records the training sessions, test session, fitting and test sample counts, excluded incomplete rows, accuracy, classification report, and confusion matrix in data/evaluation.json. [3]

Meaning of the reported metrics

Metric	Interpretation
Accuracy	Correct held out predictions divided by all held out examples
Precision for one class	Of predictions assigned to that class, the fraction correctly assigned
Recall for one class	Of true examples of that class, the fraction correctly identified
F1 score	Harmonic mean of that class precision and recall
Support	Number of true held out examples for a class
Confusion matrix	Rows are true classes; columns are predicted classes

The class order is rock, paper, scissors. For example, the true rock row and predicted paper column count rock examples mistaken for paper. Per class values help identify a weak gesture even when overall accuracy appears acceptable. [3,11]

Limits on interpretation

After evaluation, a separate live SVM is fitted on all complete sessions. The displayed holdout result belongs to the earlier evaluation model, not an independent test of that refitted model. Repeatedly changing the system against the same holdout also weakens its evidential value. A genuinely new session, and eventually new participants, are needed to evaluate improvements. A single personal session is not evidence of general population accuracy.

8 Game fairness and temporal validation

Commitment before recognition

At round start, secrets.choice selects the computer move once from the three game classes. The application commits this move before the three second countdown and hides it until a valid round is resolved. Repeated Play actions during an active round do not replace the selection. This prevents the program from observing the accepted player move and then choosing a winning response. It is a source level fairness rule, not a cryptographic commitment protocol. [5]

The opponent is random rather than adaptive. It does not use previous player choices to predict strategy. Standard rock paper scissors rules determine the result: rock beats scissors, scissors beats paper, paper beats rock, and equal moves draw. The SVM has no role in opponent selection.

Temporal stability gate

A prediction from one frame can flicker as fingers move or landmarks change. The controller therefore keeps a rolling observation window of up to 0.70 seconds. A valid stable label requires all the following conditions. These thresholds are implemented settings rather than values optimized in a formal study. [5]

- At least three observations occur in the window.
- The observations span at least 0.35 seconds.
- The most common valid label occupies at least 75 percent of all observations in the window.
- The current observation agrees with that label; a missing hand cannot reuse an old stable move.

The stability bar summarizes temporal consistency. It is neither test accuracy nor calibrated classifier confidence. This distinction is important because repeated agreement can occur even when a classifier is systematically wrong.

Round timing and match completion

State or event	Behavior
Start	Commit computer move; begin three second countdown
Stable move at or after countdown	Resolve against committed move within the allowed deadline
No stable move at countdown end	Allow up to 1.5 seconds of extra settling time
Uncertainty beyond the deadline	Cancel the round without scoring
Match	First to three wins; draws and canceled attempts are not decisive rounds

Best of five refers to decisive rounds, so total attempts can exceed five. Navigating away during a round cancels it without scoring, and New match resets the match state. Camera loss stops capture and cancels an active round. These explicit transitions make interruptions understandable and prevent stale observations from being treated as new results. [2,5]

9 Interface and implementation behavior

VisionPlay uses an OpenCV window with three main screens. The interface combines buttons and keyboard shortcuts so the demonstration can continue even when the user prefers one input method. Rendering and hit regions are supplied by ui.py, while controller state determines which actions are meaningful at that point. [2,5]

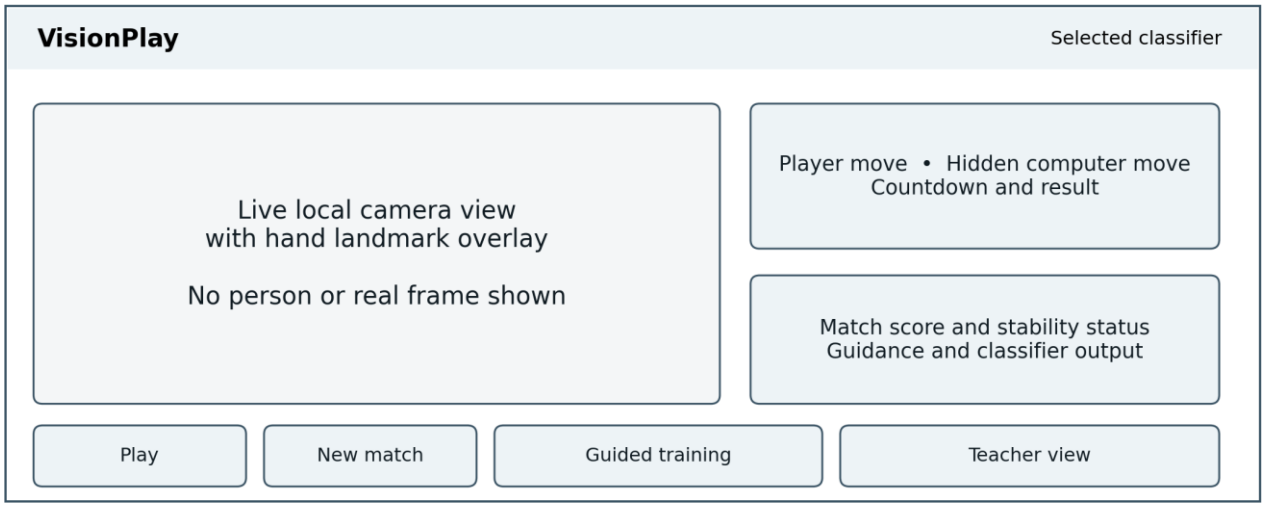


Figure 3 Illustrative interface layout. This schematic is not an exact screenshot and contains no observed scores or camera image.

Screen responsibilities

The game screen presents the camera frame, hand overlay, move icons, match score, countdown, result, stability status, and classifier information. The computer move stays hidden while the round is active. The displayed frame rate is a live interface indicator; one reading is not a controlled performance benchmark.

The training screen shows the requested pose, batch count, session progress, preparation and capture state, and controls to pause, continue, or train. Explicit stage changes reduce accidental labeling during a transition from one pose to another. The teacher screen explains the processing stages and displays actual SVM evaluation when available. An empty evaluation state must remain empty rather than show a demonstration number. [2–5]

Feedback and interruption handling

Guidance about a hand being small or near the frame edge comes from landmark geometry. It is not a detailed diagnostic model of lighting, camera quality, or user ability. The guidance helps a user adjust the scene, but does not establish why every recognition failure occurred.

Mode changes are restricted during an active round. Navigation cancels the active round without scoring and preserves already captured training rows. If a camera read fails, the application stops capture, cancels the round, reports an error, and releases resources instead of showing an old frame as fresh evidence. The camera, recognizer, and display windows are cleaned up on exit. [2,5]

10 Setup reproducibility and privacy

Supported project environment

The Windows package targets 64 bit Python 3.10 to 3.12 and prefers Python 3.11. This is the support range of the pinned project, not a claim about every newer release of its libraries. The four pinned top level dependencies are MediaPipe 0.10.21, NumPy 1.26.4, opencv-contrib-python 4.11.0.86, and scikit-learn 1.6.1. OpenCV is imported in Python as cv2. [1]

1. Download and extract the whole project to a normal writable folder. Do not start the BAT files from inside a ZIP archive.
2. Ensure a supported 64 bit Python interpreter is installed. Keep other Python versions if needed; a virtual environment cannot install a missing base interpreter.
3. Run setup.bat with internet access. It validates or creates .venv, installs the pinned dependencies, downloads the official model with a checksum check, and performs a model smoke test.
4. Run runme.bat, allow camera access, and focus the video window. Test a real round before class. To try camera index 1, run runme.bat 1 from Command Prompt.

Setup does not require an account or API key and does not silently install a global Python interpreter. A valid project environment is reused. If an old environment is incompatible, preserve it by renaming it before rerunning setup; do not delete captured data as a troubleshooting shortcut. [1,13]

Essential controls

Control	Action
Space	Play on game screen; capture on training screen
G and C	Open training; capture the current stage there
T and E	Train on training screen when capture is inactive; open teacher view
M and R	Switch available classifier outside a round; new match
Escape and Q	Return or pause from another screen; quit

Local data and privacy

Normal inference runs locally after setup. Webcam frames stay in memory; the application does not implement image recording or uploading. Optional training saves numerical features and labels in data/samples.csv and a metrics report in data/evaluation.json. A live SVM is rebuilt in memory from sufficient valid saved data on startup; no serialized SVM is shipped. [1–3]

Landmark records should still be treated as user data. For classroom use, keep the folder under the team's control and obtain agreement before collecting another person's samples. This report includes original diagrams rather than personal camera screenshots. These practices limit unnecessary disclosure without claiming that numerical hand features are anonymous.

11 Testing and available verification

Evidence categories

Software correctness and recognition quality answer different questions. A test can establish that a canceled round leaves the score unchanged without proving that scissors is recognized accurately. This report therefore separates source inspection, the historical verification record, the saved local SVM evaluation, the user's working status, and measurements that are still absent.

Evidence	Available finding	What it does not establish
Source audit on 27 October 2026	23 repository files matched revision 415ef64	Successful execution on every laptop
Published verification from 26 October	32 automated tests recorded as passed	Freshly rerun tests for this report
Headless integration in prior record	Navigation, interruption and cleanup checked using stubs	Physical webcam behavior or gesture accuracy
Native model smoke test in prior record	Official model loaded and processed a blank frame	Correct recognition of real hand poses
User report on 27 October	Application reported working after troubleshooting	Accuracy, latency, FPS or exhaustive acceptance
Local SVM evaluation dated 27 October 2026	Six complete sessions; 720 rows; saved holdout 120/120 correct	Independent participants, live-model score, or pretrained-path accuracy

The published record also documents dependency checks, Python compilation, and independent rendered UI review. These are historical build results, not measurements rerun during report preparation. An earlier webcam version was tested on Windows; the repository’s own record states that the polished interface had not received a physical Windows webcam acceptance test at that point. The later user confirmation supports basic operability but does not replace a documented test matrix. [6]

Recommended acceptance check on the presentation laptop

- Verify camera access and each of the three poses in the actual room and lighting.
- Start a round, confirm the hidden computer move, resolve a stable gesture, and reset the match.
- Remove the hand during a round and confirm that uncertainty does not produce an unintended score.
- Open training, capture and pause a batch, and confirm saved progress is retained.
- Open teacher view and confirm that values match the real evaluation file, or that no measured result is shown when training is unavailable.

Record the tested revision, operating system, interpreter, camera, date, and observations. A failed check should be described with its reproduction steps rather than silently removed from the report. This acceptance procedure is proposed work unless the team records its completion; no unchecked item is claimed as passed here.

12 Results and discussion

Observed outcome

The available evidence supports a functioning educational application with two selectable classification paths, a guided training workflow, a session holdout evaluator, and explicit round timing. The user confirmed that it was working on 27 October 2026. The local samples.csv contains 720 finite landmark rows across six complete capture sessions, with 240 rows per class. The saved evaluation.json records 600 fitting rows and 120 held-out rows and reports 100.0% accuracy, or 120 correct predictions out of 120. Its class counts and confusion matrix agree with the CSV grouping. The classifier was not independently refitted for this report, and no controlled timing or pretrained-path accuracy test is available. [3,6,14]

Saved local evaluation results

The following values come from the local evaluation record dated 27 October 2026 and its associated CSV. The newest of six complete sessions was held out; the preceding five were used to fit the evaluation model. These figures describe this specific session split. They do not measure accuracy on new people, establish independence of capture conditions, or score the final live model refitted on all complete sessions. [3,14]

Measure	Saved local evaluation, 27 October 2026
Complete sessions and excluded rows	6 complete sessions; 0 excluded rows
Evaluation fitting samples and held out samples	600 fitting rows; 120 held-out rows
Held out SVM accuracy	100.0% (120/120), saved session holdout
Rock precision recall F1 and support	Precision 1.00; recall 1.00; F1 1.00; support 40
Paper precision recall F1 and support	Precision 1.00; recall 1.00; F1 1.00; support 40
Scissors precision recall F1 and support	Precision 1.00; recall 1.00; F1 1.00; support 40
Pretrained path recognition accuracy	Not independently measured
End to end latency and sustained FPS	Not benchmarked

Confusion matrix from the saved session holdout. True classes form rows and predicted classes form columns. [14]

True class	Predicted rock	Predicted paper	Predicted scissors
Rock	40	0	0
Paper	0	40	0
Scissors	0	0	40

13 Limitations future work and conclusion

Limitations

The saved dataset contains 720 rows, but its session identifiers do not document distinct participants, cameras, lighting, or capture conditions. Session separation reduces leakage from neighboring frames but does not prove generalization to new people or settings. Human labeling errors remain possible because the wizard trusts the requested pose. The reported 100.0% applies only to the recorded 120-example session holdout, not to all future gestures or the refitted live model. [3,14]

Recognition depends on the pretrained hand detector and landmark estimator. Occlusion, motion, edge cropping, pose rotation, and unusual viewpoints can affect both paths. Geometric normalization reduces translation and apparent scale variation but does not solve all of these changes. The SVM has no unknown class, so an unfamiliar pose may still be classified as a valid move. Temporal agreement reduces flicker but cannot correct a consistently wrong label.

Only one hand is considered. No independent participant accuracy study, matched comparison with the pretrained path, or controlled runtime benchmark is included in the available evidence. The application is a desktop educational demonstration, not a validated accessibility product or a general sign language recognition system.

Prioritized future work

1. Preserve the current numerical samples, session identifiers, evaluation JSON, source revision, and any available capture conditions. Repeat evaluation on genuinely new sessions before revising the reported performance claim.
2. Collect genuinely new sessions and participants with permission. Keep participant or session groups separate when evaluating broader generalization.
3. Investigate an unknown gesture class or a separately validated rejection strategy. Evaluate rejection and misclassification together instead of treating every output as certain.
4. Compare candidate feature representations, including orientation handling, using a separate tuning process and an untouched final test set. Do not repeatedly optimize against the current holdout.
5. Measure frame processing latency and sustained throughput under specified hardware and camera conditions. Include uncertainty, rejected rounds, and failure cases in usability assessment.

Conclusion

VisionPlay demonstrates how computer vision, pretrained deep learning, and a small supervised classifier can be connected to an interactive application. Its main educational strengths are the visible landmark representation, explicit labeling workflow, session based evaluation, and separation of recognition from fair game logic. Local processing and limited stored data also keep the demonstration straightforward to operate.

The source and available verification support the implemented workflow, and the user's confirmation supports basic operability. The saved evaluation supports a narrower numerical statement: 120 of 120 held-out examples were classified correctly in one recorded session split. It does not establish accuracy for new people, an independent score for the refitted live SVM, or superiority over the pretrained path. Future evaluation should record capture conditions, add new participants and sessions, and report failures as well as successes.

References

Project sources are pinned to revision 415ef64bc8e8473a80c5959891d663dc7b9bcd0c. Official documentation was consulted on 27 October 2026. The scikit-learn references use version 1.6 to match the pinned project dependency. Links below are clickable in the editable document and PDF.

[1] VisionPlay Project. README and project setup documentation. README.md; requirements.txt; setup.bat

[2] VisionPlay Project. Frame processing and feature implementation. app.py and core.py

[3] VisionPlay Project. Training pipeline and session based evaluation. learning.py

[4] VisionPlay Project. Guided training state and capture behavior. training_state.py

[5] VisionPlay Project. Game states and temporal stability. game_state.py and controller.py; ui.py

[6] VisionPlay Project. Build verification record dated 26 October 2026. VERIFICATION.txt

[7] Google AI Edge. Gesture recognition guide for Python.

[8] Google AI Edge. Gesture recognition task guide.

[9] scikit-learn 1.6.1. SVC API reference.

[10] scikit-learn 1.6.1. StandardScaler API reference.

[11] scikit-learn 1.6.1. Metrics and scoring.

[12] OpenCV. OpenCV Python tutorials.

[13] Python Software Foundation. Python 3.11 virtual environment documentation.

[14] VisionPlay Project. Local saved dataset and evaluation dated 27 October 2026. data/samples.csv; data/evaluation.json. Aggregate results only; raw rows are not reproduced.

Evidence and attribution notes

The source audit compared 23 repository files with revision 415ef64 and found matching Git blob hashes. That audit is a source integrity check, not an execution benchmark. The working status is based on the user's confirmation on 27 October 2026. The aggregate SVM figures come from local evaluation.json and were checked against the CSV counts and recorded matrix; the fitting step was not independently rerun for this report. No private camera screenshot or raw personal dataset is reproduced here. [14]

The project README credits smoorjani/rock-paper-scissor as concept inspiration and describes VisionPlay as a new implementation. No performance figures from that repository are used here. Diagram content in this report was drawn from the audited VisionPlay code; the diagrams are explanatory rather than measured output.